CONTINUATION-IN-PART: Abstract GT I/O and Network Addressing

Church-Turing Meta-Machine: Hardware-Routed Abstract Capability Tokens for Unified I/O, Network Tunneling, and Guarantee-Based Service Scoping

Inventor: Kenneth James Hamer-Hodges

Filing Date: March 2026

Parent Application: Church-Turing Meta-Machine: Hardware-Enforced Capability Security Through Dual Trusted Gates, Lambda Calculus Integration, and Architectural Vulnerability Elimination (Filed February 2026)

Classification: Computer Architecture; Hardware Security; Capability-Based Computing; I/O Virtualization; Network Security; Service-Oriented Architecture; Deterministic Garbage Collection; Domain Separation

TITLE OF THE INVENTION

Church-Turing Meta-Machine: Abstract Golden Token I/O and Network Addressing Architecture Enabling Hardware-Enforced Guaranteed Crime-Free Business Services Through Structural Capability Scoping by Profession, Language, Nationality, and Age

CROSS-REFERENCE TO RELATED APPLICATIONS

This continuation-in-part extends the CTMM patent applications filed February 2026, which disclosed:

1. The Golden Token capability architecture and dual-gate trusted security base (mLoad / mSave)
2. Domain purity enforcement separating Turing-domain (R, W, X) from Church-domain (L, S, E) permissions
3. The LAMBDA instruction for lightweight in-scope code application
4. The atomic abstraction architecture and deterministic garbage collection (G-bit mechanism)
5. The Pure Church Lambda Machine, demonstrating computational completeness through exclusive Church-domain instructions

The present application extends that disclosure with:

1. **Abstract Golden Tokens** — a novel capability token class (`gt_type = 11₂`) whose `word1_location` field holds a hardware-routed sentinel address instead of a namespace slot index
2. **The Abstract Address Space** — a 32-bit reserved range (0xFE000000–0xFFFFFFFF) controlled exclusively by the IDE for I/O peripherals, network tunnels, and system resources
3. **The Home Base Tunnel** (0xFF000000) — the single outbound network gateway through which all CTMM network connectivity flows, with optional programmer-defined backup IDE addresses (Word 2/3)

4. **MTBF Qualification and Downloadability Regulation** — hardware-tracked reliability metrics that gate whether abstractions may propagate beyond their provisioning c-list, with three tiers: Isolated (local only), User-regulated (individual distribution), Namespace-regulated (full namespace access)

5. **Local Peripheral Autonomy** — CTMMs identify and secure locally attached hardware (UART, GPIO, Timer, Display) without any IDE connection, enabling air-gapped and offline operation

6. **Guaranteed Crime-Free Business Services** — structural capability scoping (not policy-based filtering) that prevents access to out-of-scope services without any bypass path, enabling verifiable safety for professional, language-specific, jurisdictional, and age-appropriate service isolation

7. **Secure Individuality of Abstractions** — each abstraction's identity is its unique GT set in its c-list, eliminating the "privileged superuser" attack window and preventing confused deputy attacks entirely

FIELD OF THE INVENTION

The present invention relates to a processor architecture that provides a unified capability-token-based mechanism for hardware-routed I/O and remote network access, eliminating the need for separate I/O subsystems, device drivers, or network protocol stacks. The architecture enables the IDE to establish deterministic, verifiable service scoping at the architectural level — not through runtime policy enforcement, but through structural capabilities — such that a user's CTMM provisioned for a specific profession, language, jurisdiction, or age group cannot access any service outside that scope, regardless of what code runs on the machine.

BACKGROUND OF THE INVENTION

The I/O Problem

Contemporary processor architectures separate I/O from computation through a separate subsystem: an I/O controller, device drivers, and operating system I/O stacks. This separation creates multiple security boundaries:

1. **I/O Controller Privilege** — The I/O controller runs privileged firmware that any code on the CPU can request, creating a confused-deputy vulnerability. Malicious code can request the I/O controller to access any peripheral it manages.

2. **Device Driver Complexity** — Device drivers are privileged code (typically millions of lines) that perform untrusted I/O operations on behalf of applications. Buffer overflows in drivers escalate to kernel privilege.

3. **Global I/O Namespace** — All I/O resources are accessed by name (e.g., `/dev/uart0`, `/dev/gpio5`), shared across all processes. No isolation between applications or users.

4. **Network Subsystem as Privileged Intermediary** — All network access routes through a privileged TCP/IP stack. The "network" is a privileged black box that applications cannot directly control or verify.

The Service Scoping Problem

In contemporary systems, service scoping is enforced through **policy layers** running on top of a privileged substrate:

- **Content filters** (block access to non-professional content) run on the privileged network stack.
- **Age gates** (block adult content) run on the privileged application server.
- **Jurisdiction checks** (data residency) run on the privileged cloud infrastructure.
- **Language routing** (direct to the right service) runs in privileged DNS/load-balancer logic.

Every one of these policies is a **filter applied after the capability to access the resource is granted**. Filters can be bypassed by:

- VPN and proxy services (defeat jurisdiction checks)
- DNS spoofing and man-in-the-middle (defeat language routing)
- Malicious extensions and injected scripts (defeat content filters)
- Buffer overflow in the filter itself (defeat age gates)

No conventional system provides a **structural guarantee** that access is impossible without the right token.

The Trusted Network Gateway Problem

In contemporary architectures, the network is accessed through a single privileged gateway (the TCP/IP stack, the hypervisor's network device, the cloud provider's edge router). If this gateway is compromised — or if the entity controlling it becomes adversarial — all network access can be monitored, throttled, or redirected. There is no way for the user or the CTMM to detect or prevent this.

The Discovery

The parent CTMM application's Abstract GT type field opens a new possibility: a capability token that is not a namespace reference, but a **hardware-routed sentinel address**. This token type can be provisioned by the IDE exclusively at boot time — before any user code runs — and cannot be forged or synthesized by software.

By extending this principle to a full Abstract Address Space, the CTMM can:

1. **Make I/O resources unforgeable capabilities** — no code can access a peripheral without holding the Abstract GT for it
2. **Make network access an unforgeable capability** — the Home Base tunnel (0xFF000000) is the only outbound connection; all network access flows through it
3. **Enable structural service scoping** — a child CTMM provisioned without the GT for adult services has no path to them, regardless of what code it runs
4. **Enable local autonomy** — peripherals are identified and secured during local hardware boot, not by IDE decree
5. **Enable offline operation** — local services (UART, GPIO, storage) work with no IDE connection; the Home Base tunnel is optional

THE ABSTRACT ADDRESS SPACE: A NEW RESERVED CAPABILITY DOMAIN

What Is an Abstract GT?

An Abstract Golden Token is a 128-bit capability register with:

```
    Word 0 (32-bit GT):
  [15:0]  object_id / slot_id (not used for NS lookup — sub-identifier within Abstract range)
  [22:16] gt_seq (not used — zero for Abstract GTs)
  [24:23] gt_type = 11₂ (Abstract)
  [30:25] permissions (R, W, X, L, S, E)
  [31]    b_flag (may be propagated via mSave)

Word 1: word1_location (32-bit Abstract Address — the hardware-routed sentinel)
Word 2: word2_backup1 (first backup Abstract Address; 0x00000000 = not configured)
Word 3: word3_backup2 (second backup Abstract Address; 0x00000000 = not configured)
```

Unlike Inform GTs (which point to namespace entries) and Outform GTs (which point to remote resources), an Abstract GT's `word1_location` is **not dereferenced**. Instead, it is matched directly against the Abstract Address Space table in hardware. The address is the token's identity.

The Abstract Address Space Layout

The 32-bit `word1_location` range is reserved exclusively for the IDE:

```
    0x00000000 — 0xFDFFFFFF    Real RAM — never an Abstract GT address
0xFE000000 — 0xFEFFFFFF    Local hardware peripheral range (64K entries)
                            UART, GPIO, Timer, Display, Storage identified by
                            CTMM during local hardware boot
0xFF000000                 Home Base tunnel — primary outbound network gateway
0xFF000001 — 0xFF0000FE    IDE-allocated tunnel channels (254 named remote services)
                            Each is a distinct encrypted tunnel to a named endpoint
0xFF0000FF — 0xFFFEFFFF    Reserved for future IDE-defined Abstract resources
0xFFFF0000 — 0xFFFFFFFD    Reserved for future system Abstract GTs
0xFFFFFFFE                 SWITCH PassKey for CR13 (IRQ Thread) — from Task #58
0xFFFFFFFF                 SWITCH PassKey for CR15 (Namespace) — from Task #58
```

Critical property: Abstract Addresses are not stored in the namespace table. There is no CRC validation, no namespace lookup, no lump header dereference. The address alone is the capability.

Unforgeability

Software cannot construct an Abstract GT with a reserved address because:

1. Only the IDE (running at boot, before user code) can write directly to capability registers
2. After boot, Abstract GTs can only be copied (via mSave, if b_flag=1), attenuated (via TPERM, removing permissions), or nullified
3. No instruction allows synthesis of a GT from components

A program that was not given a GT for a resource cannot obtain one — period.

THE HOME BASE TUNNEL: UNIFIED NETWORK GATEWAY

What It Is

The Home Base Tunnel (Abstract Address 0xFF000000) is an Abstract GT that represents the CTMM's outbound connection to the IDE and cloud infrastructure. It is the **sole network interface** through which all CTMMs communicate with the outside world.

The Home Base GT is provisioned at boot by the IDE with:

- `word1_location = 0xFF000000`
- `perms` typically = R | W | E (receive, send, RPC invoke)

- `word2_backup1` and `word3_backup2` = programmer-defined fallback IDE addresses (optional)

How It Works

Operations on the Home Base GT route directly to the hardware tunnel driver:

Permission	Operation	Meaning
R (Read)	DREAD / mLoad	Receive data from Home Base (IDE push, config, MTBF thresholds)
W (Write)	DWRITE / mSave	Send data to Home Base (telemetry, audit logs, MTBF counters)
E (Enter)	CALL	Invoke a named remote service via encrypted RPC tunnel

Security

The Home Base is unforgeable:

- Only the IDE can write it at boot
- Code without the Home Base GT has no path to the network
- The b_flag controls whether the GT may be propagated (default: 0, not propagable)
- Revoking network access is instant: set the Home Base GT to NULL in the c-list

Programmer-Defined Backup IDEs (Novel)

A programmer can configure up to two backup IDE addresses in Word 2 and Word 3:

```
   Primary:   word1_location = 0xFF000000 (main cloud IDE)
Backup #1: word2_backup1 = 0xFF000001 (developer's private server)
Backup #2: word3_backup2 = 0xFF000002 (team fallback)
```

Hardware implements failover: if the primary is unreachable, try Backup #1; if that fails, try Backup #2. All three are provisioned atomically at boot — user code cannot change them.

Novel advantage: A developer can ensure their CTMM falls back to a trusted private IDE, not a compromised public one, without any degradation in security. The backup addresses are unforgeable, hardware-validated, and immutable.

LOCAL PERIPHERAL SECURITY: AUTONOMOUS OPERATION

The Core Insight

Each CTMM can identify and secure locally attached equipment entirely on its own — without any IDE connection, network access, or remote authority.

During the CTMM's hardware boot sequence (before any software runs):

1. The hardware probe enumerates attached peripherals (UART, GPIO, Timer, Display, Storage)

2. For each peripheral, the boot sequence assigns it an Abstract Address from the local range (0xFE000000+)

3. The boot sequence creates an Abstract GT for each peripheral and provisions it into the appropriate c-list

4. All of this happens **offline, locally, before the IDE is even contacted**

Security Advantages

Scenario	Conventional System	CTMM with Local Autonomy
Air-gapped operation (no network)	I/O locked until network available	All local I/O fully functional
Offline mode	Peripherals inaccessible without cloud	Peripherals secured locally with full GT enforcement
Malicious IDE	IDE controls peripheral access; IDE can deny all I/O	IDE did not provision local peripherals; CTMM controls them
Autonomous agent in remote location	Must phone home for every I/O operation	Operates independently for all local resources

CTMM is the Authority for Its Own Hardware

The security decision — "which abstractions receive the peripheral GTs, with what permissions" — is made by the **local boot policy, not by any remote party**. A remote IDE has no ability to:

- Revoke access to a locally attached UART
- Deny GPIO access
- Block storage operations
- Isolate the CTMM from its own hardware

This is a fundamental shift from conventional architectures, where the OS (a privileged entity) mediates all I/O access.

MTBF QUALIFICATION AND DOWNLOADABILITY REGULATION

The Problem

When an abstraction is propagated from one CTMM to another via mSave, the receiving CTMM accepts it without question. But what if the abstraction is unreliable? What if it crashes frequently, corrupts data, or is malicious?

Contemporary systems rely on **trust relationships** (code signing, sandboxing, user reputation) to decide what to accept. All of these can be forged or manipulated.

The Solution: Hardware-Tracked MTBF Qualification

Every Secure Abstraction carries a hardware-tracked **MTBF qualification** — a reliability metric that determines whether it may be distributed beyond its local CTMM and, if so, to whom.

The hardware tracks two counters per abstraction in the namespace entry:

- `invocation_count` — number of times the abstraction has been called
- `failure_count` — number of those calls that resulted in a FAULT or exception

```
MTBF score = invocation_count / (failure_count + 1)
```

Three Downloadability Tiers

Tier	MTBF Condition	S Permission	Scope

Isolated	Below threshold or unvalidated	Hardware-locked S=0	Local CTMM only; cannot propagate
User-regulated	Meets user-tier MTBF	S enabled	Individual user distribution via Home Base tunnel
Namespace-regulated	Meets namespace-tier MTBF	S enabled	Full namespace access; CR15 validates each download

IDE Control of Thresholds

The parent IDE sets and updates the MTBF thresholds remotely via the Home Base tunnel:

```
   Threshold payload (signed by IDE):
isolated_floor = 0.90          — MTBF score below this locks S=0
user_tier_threshold = 0.99     — score above this unlocks S for user distribution
ns_tier_threshold = 0.999      — score above this unlocks namespace distribution
min_invocations = 100          — don't qualify until at least 100 invocations
```

The IDE can raise thresholds (tighten quality requirements) or lower them without firmware updates. Thresholds take effect on the next invocation cycle.

Telemetry and Trust

The CTMM sends MTBF telemetry (counters + timescale data) back to the IDE via Home Base W permission, signed with HMAC-SHA256. The IDE maintains a **permanent MTBF record per abstraction per CTMM**, creating a distributed reputation system:

- An abstraction that fails on one CTMM (low MTBF score) becomes less trustworthy on all CTMMs through IDE policy update
- A highly reliable abstraction (high MTBF score) qualifies for namespace distribution, allowing others to receive and use it
- A freshly installed, unvalidated abstraction starts at Isolated tier and must earn its way to User or Namespace tier through demonstrated reliability

GUARANTEED CRIME-FREE BUSINESS SERVICES

The Core Idea

The Abstract GT architecture enables a fundamentally new class of service provider: one that can **guara ntee** — not promise, but **guarantee** — that its service is crime-free. This is structural, not policy-based.

Why "Guaranteed"

In conventional systems, crime prevention relies on **policy layers** (filters, gates, rules) running on a privileged substrate that can itself be compromised:

- A content filter can be bypassed if the filter code is exploited
- An age gate can be defeated by VPN or cookie manipulation
- A jurisdiction check can be circumvented by spoofed origin headers

The Church Machine achieves guarantees through **structural capability scoping**:

> A CTMM provisioned without a GT for a service cannot access it, regardless of what

code runs on the machine or what exploits are discovered.

There is no filter to bypass, no gate to trick, no policy to manipulate. **The capability does not exist.**

Scoping by Profession

Each professional domain is a distinct GT set:

```
   Medical Professional CTMM holds:
- GT for medical database (namespace reference)
- GT for clinical reference service (tunnel channel)
- GT for regulated comms (tunnel channel)
- NO GT for financial services
- NO GT for adult content
- NO GT for gambling services
```

A medical professional's software simply has no path to financial trading platforms, legal databases, or gambling services. Not because of a policy, but because the GT doesn't exist.

Scoping by Language

Language routing is a GT boundary:

```
   French-language service abstraction holds:
- GT for French service endpoint (tunnel channel 0xFF000001)
- GT for French data repository (namespace)
- NO GT for English service endpoint
```

An abstraction provisioned for French-language service cannot reach English-language equivalents. The capability is absent.

Scoping by Nationality and Jurisdiction

Data residency and legal boundaries are GT boundaries:

```
   EU CTMM holds:
- GT for EU data center (tunnel channel)
- GT for GDPR-compliant services (tunnel channel)
- NO GT for US-only services
- NO GT for non-GDPR endpoints
```

An EU-provisioned CTMM cannot reach non-GDPR-compliant services because it was never given the GT for them. The IDE enforces this at provisioning time, and hardware enforces it at runtime.

Scoping by Age Group

Child-safe operation is structural:

```
   Child CTMM (age 8) holds:
- GT for educational content services
- GT for parental-approved websites
- GT for homework-helper endpoints
- NO GT for adult content services
- NO GT for social media (unmoderated)
- NO GT for financial/gambling services
- NO GT for unvetted remote services
```

A child's CTMM has no capability to reach adult services. Injected JavaScript cannot conjure a GT. Malicious links cannot create tunnels. VPN tools cannot bypass this — the capability is missing at the hardware level.

This is qualitatively different from any filter-based child protection system. A filter examines content after the child already has the capability to fetch it. The Church Machine prevents the capability from existing in the first place.

SECURITY PROPERTIES AND HARDENING ROADMAP

Phase 1 Hardening (Immediate, Non-Silicon)

To address identified security gaps, the following Phase 1 mitigations are non-blocking and ready for implementation:

1. **Home Base Threshold Signing** — MTBF threshold payloads are signed by IDE public key; CTMM validates signatures before installing
2. **Immutable Threshold Ledger** — Every threshold change logged to append-only NVM; boot verifies current threshold matches latest ledger entry
3. **MTBF Snapshot Validation** — Counters are captured in NVM on first abstraction run; reset attempts are detected and flagged as Suspicious
4. **CRC Failure Watchdog** — CRC validation failures per NS entry tracked; Poisoned entries FAULT on all access after 3 failures
5. **Rate-Limiting on mLoad/SWITCH** — Max 3 contention retries per instruction before TRAP, preventing timing side-channel attacks
6. **Backup Address Validation** — Hardware ensures Word 2 ≠ Word 1, Word 3 distinct from both, with recently-tried queue preventing routing loops
7. **Chain-of-Custody Logging** — Every mSave propagation logged with sender, recipient, timestamp for forensics
8. **Wraparound Safety** — Minimum GC interval prevents rapid gt_seq wraparound exploitation

Phase 2 Hardening (Full Architecture)

Comprehensive fixes requiring hardware/NS layout changes:

1. **Dual-Root Home Base Trust** — Two independent Home Base GTs, both must sign policy (2-of-2 threshold)
2. **Tamper-Proof MTBF Counters** — Hardware-owned, non-writable except by CALL/RETURN microcode
3. **Leased GT Validity** — Time-limited GT validity (TTL) for automatic revocation of propagated copies
4. **Cryptographic Integrity** — HMAC-SHA256 or dual CRC-32 replacing 16-bit CRC
5. **Per-Recipient Revocation** — Bloom filter in NS entry enabling selective revocation of propagated GTs

DETAILED CLAIMS (NOVEL)

Claim 1: Abstract Golden Token with Hardware-Routed Sentinel Address (Independent)

A capability register architecture wherein a Golden Token of type `Abstract` (`gt_type = 11₂`) contains a `wo`

`rd1_location` field that is a 32-bit sentinel address outside the real RAM range, and wherein hardware matches this address directly against an Abstract Address Space table rather than performing a namespace lookup, such that the address alone constitutes the token's identity and unforgeability, without any namespace slot allocation, CRC validation, or lump header dereference.

Claim 2: Home Base Tunnel as Single Network Gateway (Independent)

An I/O virtualization mechanism wherein a single Abstract GT (Home Base tunnel at address 0xFF000000) is the sole outbound network interface, with all network access from all abstractions routed through this single hardware-managed tunnel, and wherein the Home Base GT can be provisioned with optional programmer-defined backup IDE addresses (Word 2 and Word 3) that are tried in sequence if the primary address is unreachable, with all three addresses set atomically at boot and immutable thereafter.

Claim 3: Local Peripheral Autonomy Without IDE Assistance (Independent)

A hardware bootstrap mechanism wherein attached peripherals (UART, GPIO, Timer, Display, Storage) are identified and Abstract GTs are provisioned entirely during local hardware boot, before any user code runs and before any IDE connection is established, such that the CTMM maintains full security control over local I/O without any remote authority, enabling air-gapped, offline operation where the CTMM is the sole authority for its own hardware security policy.

Claim 4: MTBF Qualification as Hardware-Enforced Downloadability Gate (Independent)

A hardware-tracked reliability mechanism wherein every Secure Abstraction carries `invocation_count` and `failure_count` counters, an MTBF score is computed as `invocation_count / (failure_count + 1)`, and the S (Save) permission on the abstraction's GT is hardware-locked based on the MTBF tier (Isolated, User-regulated, Namespace-regulated), such that unreliable abstractions cannot propagate beyond their provisioning context, and the IDE remotely updates tier thresholds via signed policy payloads delivered through the Home Base tunnel.

Claim 5: Structural Capability Scoping for Crime-Free Services (Novel)

A service provisioning architecture wherein a CTMM provisioned for a specific profession, language, nationality, or age group receives only Abstract GTs for services within that scope, such that any code running on the CTMM cannot access services outside the scope because the capability token does not exist, providing a structural guarantee (not a policy-based filter) that access is impossible regardless of exploits, VPN tunnels, or code injection — because the hardware rejects any operation without a valid GT.

Claim 6: Two-Tier Backup IDE Fallback with Atomic Provisioning (Dependent)

An extension of Claim 2 wherein the Home Base GT includes `word2_backup1` and `word3_backup2` fields, each containing a 32-bit Abstract Address within the tunnel range, and hardware implements sequential failover: try primary → try Word 2 if reachable → try Word 3 if reachable → TRAP: TUNNEL_UNAVAILABLE, with loop detection ensuring no two addresses are identical and no recently-tried address is re-attempted within the same connection attempt.

Claim 7: MTBF Telemetry and Distributed Reputation (Dependent)

An extension of Claim 4 wherein MTBF counters are sent to the IDE via Home Base W permission, signed with HMAC-SHA256, and the IDE maintains a permanent reputation record per abstraction per

CTMM, such that an abstraction's reliability history is shared across the fleet: low MTBF on one CTMM lowers the threshold for all CTMMs through policy update, and high MTBF qualifies abstractions for namespace-tier distribution.

Claim 8: Secure Individuality via Unique GT Sets (Dependent)

A trust isolation mechanism wherein each abstraction's identity is defined as the unique set of GTs provisioned in its c-list, such that no two abstractions share ambient authority, an attacker who compromises one abstraction gains only its GTs and cannot escalate to another abstraction's resources without holding that abstraction's GTs, and the "privileged superuser" attack window is eliminated entirely — there is no privileged layer that holds authority on behalf of others.

DENIAL OF SERVICE PREVENTION AND CONTROL MECHANISMS (NOVEL)

The DoS Problem in Conventional Systems

Denial of Service attacks exploit several fundamental weaknesses in contemporary architectures:

1. **Unbounded Resource Access** — Any code can request arbitrary resources (file I/O, network bandwidth, memory allocation) without proving it holds the right to them.
2. **Privilege Escalation via DoS** — A low-privilege process can exhaust system resources, forcing the OS to deny service to higher-privilege processes. The OS itself becomes the bottleneck.
3. **No Per-Resource Rate Limiting** — All access to a resource goes through a shared bottleneck (e.g., the network stack, the filesystem). An attacker can saturate the bottleneck for all users.
4. **Ambient Authority Sharing** — Multiple processes share TCP/IP stack, file descriptor table, memory allocator. An attack on any shared resource affects all processes.
5. **Cascade Failures** — When one service is DoS'd, dependent services starve. The attack propagates through the system.

How CTMM Prevents DoS

The Church Machine architecture prevents or drastically limits DoS attacks through structural mechanisms:

Mechanism 1: Capability-Gated Resource Access

Only code that holds an Abstract GT for a resource can access it. To launch a DoS attack on a resource, the attacker must first hold the capability.

```
   Example: UART Service DoS Prevention
- UART is provisioned as Abstract GT 0xFE000001 (local peripheral)
- Only abstractions with GT for 0xFE000001 can access UART
- Attacker code without the GT cannot request any UART operations
- UART DoS is impossible without holding the capability
```

Impact: An attacker in one abstraction cannot DoS a resource that abstraction was not given access to.

Mechanism 2: Per-Abstraction Isolation

Each abstraction has its own c-list with its own GT set. Resources are not ambient or shared by default.

```
      Two abstractions running concurrently:
  Abstraction A: holds GT for Service X only
  Abstraction B: holds GT for Service Y only

If A is DoS'd by local attacker:
  - B continues running normally (no shared resources exhausted)
  - Service Y remains responsive
  - Only Service X is affected

The attack is contained to the attacker's own abstraction.
```

Impact: One compromised abstraction cannot cascade-DoS other abstractions.

Mechanism 3: Hardware-Enforced mLoad Retry Limits

All namespace access routes through mLoad, which has a maximum of 3 contention retries per instruction before raising `TRAP: MAX_RETRIES_EXCEEDED`.

```
      mLoad retry sequence:
  Attempt 1: LOAD instruction retries (contention)
  Attempt 2: LOAD instruction retries again (contention)
  Attempt 3: LOAD instruction retries once more (contention)
  Attempt 4: TRAP: MAX_RETRIES_EXCEEDED (IRQ handler decides next action)
```

An attacker cannot hammer the namespace gate with unlimited retries. The attacker must explicitly wait (exponential backoff) before retrying.

Impact: Rate-limited access prevents namespace gate exhaustion.

Mechanism 4: Hardware-Enforced SWITCH Rate Limits

SWITCH instruction (privilege gate for CR13/CR15) has the same 3-retry limit as mLoad.

```
      SWITCH CR_passkey, CR15
  Attempt 1: contention, retry
  Attempt 2: contention, retry
  Attempt 3: contention, retry
  Attempt 4: TRAP: MAX_RETRIES_EXCEEDED
```

An attacker cannot force rapid privilege switching that could disrupt scheduling or timing.

Impact: Prevents privilege escalation DoS attacks.

Mechanism 5: MTBF Qualification Prevents DoS Propagation

An abstraction that crashes or fails frequently (high failure_count) is marked Isolated by hardware and cannot be propagated to other CTMMs.

```
      Malicious abstraction crashes 50% of the time:
  MTBF score = 100 / 50 = 2.0
  Threshold for User-tier: 0.99
  Score 2.0 > 0.99 → Passes user-tier threshold initially

After 1000 invocations, with 50% failure rate:
  MTBF score = 1000 / 500 = 2.0
  Score does not improve
  IDE updates threshold: user_tier = 0.999 (tighten)
  Score 2.0 > 0.999 → Still passes, but barely

After 10,000 invocations:
  If failure pattern holds: MTBF = 10000 / 5000 = 2.0
  IDE updates: user_tier = 0.9999
  Score 2.0 > 0.9999 → FAILS threshold
  S bit locks to 0: no further propagation
```

A DoS-prone abstraction cannot escape its origin and infect other CTMMs.

Impact: DoS attacks are quarantined to the originating CTMM.

Mechanism 6: Local Peripheral Rate Limiting

Hardware-controlled access to local peripherals (UART, GPIO, Timer, Display) can enforce per-abstraction bandwidth limits or operation counts.

```
   UART rate limiting example:
- Each abstraction holds a GT for the UART with R/W permissions
- Hardware enforces per-abstraction quota: max 1000 bytes per second
- If abstraction exceeds quota, further UART operations are deferred
- Other abstractions continue UART access at normal rate
```

Impact: Local I/O DoS attacks are rate-limited per abstraction.

Mechanism 7: Home Base Tunnel Flow Control

The Home Base tunnel (single network gateway) implements flow control at the hardware level:

```
   Home Base tunnel bandwidth management:
- Per-CTMM outbound quota: X Mbps max
- Per-abstraction outbound quota: Y Kbps max
- Per-connection timeout: Z seconds
- If quota exceeded, further sends are deferred (backpressured)
```

An attacking abstraction cannot starve other abstractions of network bandwidth because each abstraction has its own quota.

Impact: Network DoS attacks are rate-limited per abstraction.

Mechanism 8: GC Interval Safety Limits

Garbage collection has a minimum interval (e.g., 1 second) between wraparound events. If GC tries to wrap the gt_seq counter too frequently, a TRAP fires.

```
   gt_seq wraparound interval limit:
Last wraparound at time T
Next wraparound attempt at time T + 500ms
500ms < 1 second minimum
TRAP: GC_WRAPAROUND_TOO_FAST
```

An attacker cannot force rapid GC cycles to accelerate wraparound and confuse stale GTs with fresh ones.

Impact: GC-based DoS attacks are prevented.

Mechanism 9: Abstract Address Validation

Every attempt to use an Abstract GT requires hardware validation of the address. Invalid addresses are TRAP'd:

```
   Hardware validation on Abstract GT operation:
word1_location = 0x12345678  (not in reserved range)
Check: is 0x12345678 in Abstract Address Space?
No → TRAP: INVALID_ABSTRACT_ADDRESS
```

An attacker cannot cause DoS by constructing bogus Abstract Addresses.

Impact: Malformed operations fail fast without consuming resources.

Claim 9: Capability-Gated DoS Prevention (Independent)

A denial-of-service prevention mechanism wherein access to any resource is controlled exclusively

through unforgeable capability tokens, such that an attacker without the capability token for a resource cannot even request operations on that resource, and thus cannot launch a DoS attack on that resource, with the constraint being structural and enforced at the hardware level rather than through rate-limiting or filtering.

Claim 10: Per-Abstraction Resource Isolation (Independent)

A resource isolation mechanism wherein each Secure Abstraction has its own c-list with its own GT set, and resources (I/O, network, memory allocators, queues) are not shared by default, such that an attacking abstraction that exhausts its own resources affects only itself and does not cascade to other abstractions, enabling DoS containment at the abstraction boundary.

Claim 11: Hardware-Enforced Retry Limits on Critical Paths (Independent)

A rate-limiting mechanism wherein critical paths (mLoad for namespace access, SWITCH for privilege gate, GC for garbage collection) enforce a maximum of three contention retries before raising a TRAP that returns control to the IRQ handler, and wherein the hardware counter resets only on successful completion or explicit FAULT, such that an attacker cannot hammer critical paths without explicit wait periods (exponential backoff), preventing saturation DoS attacks on the Trusted Security Base.

Claim 12: MTBF Qualification as DoS Containment (Dependent)

An extension of Claim 4 wherein an abstraction that fails frequently (high failure_count) is automatically downgraded to Isolated tier and cannot be propagated to other CTMMs, such that DoS-prone or malicious abstractions cannot spread across the fleet through normal capability propagation, and fleet-wide policy updates further tighten MTBF thresholds to quarantine unreliable abstractions.

Claim 13: Per-Abstraction Bandwidth Quotas on Tunnel Access (Dependent)

An extension of Claim 2 wherein the Home Base tunnel implements per-abstraction outbound bandwidth quotas (measured in bytes per second or operations per second), and wherein abstractions that exceed their quota are backpressured rather than dropped, such that one attacking abstraction cannot saturate the shared tunnel and starve other abstractions of network access.

Claim 14: Local Peripheral Access Rate Limiting (Dependent)

An extension of Claim 3 wherein access to local peripherals (UART, GPIO, Timer, Display) is rate-limited per abstraction, either through hardware quotas or token-bucket mechanisms, such that an abstraction cannot monopolize peripheral bandwidth and DoS other abstractions' local I/O operations.

FIGURES (Proposed)

Figure A1: Abstract GT Address Space

Diagram of the 32-bit address space showing the namespace region (0x00000000–0xFDFFFFFF) for Inform GTs managed by Navana, the IDE-controlled peripheral I/O range (0xFE000000–0xFEFFFFFF), the Home Base Tunnel sentinel at 0xFF000000, and the reserved IDE system resource range (0xFF000001–0xFFFFFFFF). Annotations show the Abstract GT format (gt_type = 11_2, word1 holds sentinel address) and the hardware routing path that bypasses namespace lookup entirely. Callout boxes enumerate what the architecture eliminates: no device drivers, no I/O subsystem, no protocol stacks.

Figure A2: Home Base Tunnel

Architecture diagram showing the single outbound network gateway at address 0xFF000000. Local abstractions holding Outform GTs (F-bit set) route all network traffic through the Home Base Tunnel to the IDE, which forwards to remote namespaces. The tunnel word format is annotated: Word 0 (primary sentinel 0xFF000000), Word 1 (version/seal MAC), Word 2–3 (programmer-defined backup IDE addresses for fault-tolerant connectivity). Security properties are highlighted: no second outbound path, no address forgery.

Figure A3: MTBF Qualification Tiers

Three-tier progression diagram showing how hardware-tracked Mean Time Between Failures gates abstraction distribution. Tier 1 (Isolated): MTBF below user threshold, locked to local namespace, suitable for new/untested abstractions. Tier 2 (User-regulated): MTBF between user and namespace thresholds, individual distribution permitted. Tier 3 (Namespace-regulated): MTBF above namespace threshold, full namespace access granted. Hardware enforcement callouts: MTBF counters are unforgeable hardware registers, tier transitions are automatic, FAULT resets the counter, software cannot override qualification.

Figure A4: Local Peripheral Autonomy

Boot-time peripheral discovery diagram. During Phase 5, the CTMM probes IDE address range (0xFE_____) for locally attached hardware (UART, GPIO, SPI, Timer, Display). Present peripherals receive valid Abstract GTs; absent ones receive NULL GTs (accessing NULL triggers FAULT). Side-by-side comparison of Connected mode (IDE provisions Home Base Tunnel, remote namespaces accessible), Air-gapped mode (no IDE, Home Base = NULL, local peripherals still work), and Security guarantee (no outbound data leakage possible). The peripheral access model: abstraction holds Abstract GT in c-list → CALL routes to peripheral hardware → TPERM checks permissions → direct register access. No device driver, no kernel, no system call.

CONCLUSION

The Abstract Golden Token I/O and Network Addressing architecture represents a fundamental shift in how computer systems integrate I/O, network access, and service provisioning. By making these capabilities unforgeable, hardware-routed, and locally autonomous (for peripherals) or IDE-provisioned-at-boot (for network), the architecture enables:

1. **Structural safety**: Services can be scoped at the capability level, guaranteeing that certain code cannot access certain services because the token doesn't exist.
2. **Offline autonomy**: Local peripherals work without any IDE connection, making CTMMs suitable for air-gapped, autonomous, and edge-deployed scenarios.
3. **Trusted fallback**: Programmers can configure backup IDE addresses, enabling local-first or team-first development without compromising CTMM security.
4. **Distributed reputation**: MTBF qualification creates a mesh-style trust model where reliability is tracked and propagated, replacing centralized app stores.
5. **Crime-free guarantees**: For the first time, a computer system can structurally guarantee that a user's device cannot access prohibited services — not through filters or policies, but through the absence of capability tokens.

This invention builds on the CTMM foundation while opening entirely new application domains: edge

computing, offline-first systems, autonomous agents, and the first genuinely crime-free platforms.